

CYCLE 2

Consider a COMPANY database.

Create the following three tables in the database and insert atleast 5 records in each table.

1. EMPLOYEE

EMPLOYEE(EmpID, Ename, DOB, City, DOJ, Salary, DeptID)

2. DEPARTMENT

DEPARTMENT(DeptID, DeptName, Location, Fund)

3. DEPENDENT

DEPENDENT(DependentName, Age, Gender, EmpID, Relation_with_employee).

NOTE:

- ◆ Give appropriate Primary key and Foreign key constraints.
- ◆ Default value of Salary in the EMPLOYEE table should be 0.
- ◆ Gender in DEPENDENT table should accept either of the three values, 'Male', 'Female' or 'Others' only.

Perform the following queries.

Implementation of Aggregate functions

1. Find total number of employees in the company.
2. Find maximum salary in the company.
3. Find youngest dependent age
4. Find total salary paid to all employees who joined the company in the year 2022.
5. Find average age of dependents of employee with Employee ID 5.

Implementation of Group by, Order by and having clauses

1. Find average salary of employees in each department.
2. Count the number of dependents for each employee.
3. List employees ordered by salary (highest first).
4. List employees ordered by Date of Joining (latest first).
5. Find departments having more than 5 employees.
6. Find departments where average salary is greater than 50,000.
7. List departments with more than 3 employees, ordered by average salary.

Consider a COMPANY database with the following three tables.

1. EMPLOYEE

EMPLOYEE(EmpID, Ename, DOB, City, DOJ, Salary, DeptID)

2. DEPARTMENT

DEPARTMENT(DeptID, DeptName, Location, Fund)

3. DEPENDENT

DEPENDENT(DependentName, Age, Gender, EmpID, Relation_with_employee).

Practice of SQL TCL commands : savepoint, rollback, commit

1. Start a new transaction. Change the salary of the employee whose EmpID is 5 as 50000/-. Undo the update operation. Commit the transaction and display the changes to the table .
2. Start a new transaction. Delete a record from EMPLOYEE table. Then, insert a new record in DEPENDENT table. Undo the insert operation only at first. Then, undo the delete operation also. Commit the transaction and display the changes to the table .

CYCLE 3

Consider a COMPANY database with the following three tables.

1. EMPLOYEE

EMPLOYEE(EmpID, Ename, DOB, City, DOJ, Salary, DeptID)

2. DEPARTMENT

DEPARTMENT(DeptID, DeptName, Location, Fund)

3. DEPENDENT

DEPENDENT(DependentName, Age, Gender, EmpID, Relation_with_employee).

Practice of SQL Set Operators

1. Combines employee names and dependent names into a single list.
2. List all cities where either employees stay or departments are located including duplicates.
3. Find cities where both employees live and departments are located
4. Display the names that appear both as employee names and dependent names
5. Display the Ids of the Employees who have no dependents

Practice of SQL Join operators

1. List employee name and department name of each employee.
2. Display the name of employees working in the 'HR' Department
3. List the employees whose dependents are older than 18
4. List the name of employees with more than one dependent
5. List all employees and dependents using left join
6. List all employees and their department using right join
7. Display all employees and all departments, including employees without departments and departments without employees.
8. List the name of employees and department names of those employees that have no dependents

Practice of SQL nested queries

1. List the name of employees who earn more than the average salary of all employees
2. List the name of employees working in the department named 'HR'
3. Display the name and salary of the employees who have at least one dependent.
4. Display the name, ID and DOJ of employees who do NOT have any dependents.
5. Display the name, DeptID and DOB of employee(s) with the highest salary in the company.
6. List the Employee names who earn more than every employee in departments located in 'Delhi'
7. List the employee names who do NOT work in departments with fund less than company average fund

Practice of SQL VIEWS

1. Create a view that contains details of Employees staying in Bangalore.
2. Create a view that contains Employee basic details with their department names
3. Create a view that contains names of employees in CS department
4. Drop the created views

CYCLE 4

Consider a COMPANY database with the following three tables.

1. EMPLOYEE

EMPLOYEE(EmpID, Ename, DOB, City, DOJ, Salary, DeptID)

2. DEPARTMENT

DEPARTMENT(DeptID, DeptName, Location, Fund)

3. DEPENDENT

DEPENDENT(DependentName, Age, Gender, EmpID, Relation_with_employee).

Practice of SQL commands for creation of assertions.

1. Employee salary must be greater than 0
2. Department fund must be greater than total salary of its employees
3. Dependent age must be less than employee age
4. An employee can have at most 5 dependents
5. Each department must have at least one employee

Creation of Triggers

1. Create a trigger that prevents the entry of an employee record with salary less than 10,000
2. Create a trigger that limits maximum number of dependents per employee to 3
3. Create a trigger that reduces department fund when a new employee is added.
4. Create a trigger that prevents salary reduction
5. Whenever an employee's salary is updated, store the old and new salary details in a log table. Create a trigger.
6. Before deleting an employee record from EMPLOYEE, store it in a backup table. Create a trigger .

UI familiarization

1. Database initialization - Data insert, Data import to a database (bulk import using UI and SQL Commands).
 2. Export ER diagram from the database and verify relationships
-

CYCLE5

Consider a COMPANY database with the following three tables.

1. EMPLOYEE

EMPLOYEE(EmpID, Ename, DOB, City, DOJ, Salary, DeptID)

2. DEPARTMENT

DEPARTMENT(DeptID, DeptName, Location, Fund)

3. DEPENDENT

DEPENDENT(DependentName, Age, Gender, EmpID, Relation_with_employee).

PROCEDURES

1. Create a Procedure to Add New Employee.
2. Create a Procedure to Increase Salary of a particular Employee by a particular input amount.
3. Create a Procedure to Count Employees in a particular Department.
4. Create a Procedure to count the number of Employees with Salary Above Given Amount
5. Create a Procedure to Delete Employee Only If No Dependents
6. Create a procedure to transfer an Employee to Another Department

FUNCTIONS

1. Create a function to get number of employees in a particular Department
2. Create a function to Calculate Average Salary of a Department
3. 3. Create a function to Calculate Employee Age from DOB
4. Create a Function to get Details of an Employee with his/her Department details

Cassandra

I. Perform basic CRUD (Create, Read, Update, Delete) operations on a Cassandra table.

1. Create a keyspace named company.
 2. Show all available keyspaces
 3. Choose and use the keyspace created. I.e, company
 4. Create the following two tables
 - a. department (did, dname, fund)
 - b. employee (eid, ename, dob, deptid)
 - c. dependent (empid, dependentname, age)
 5. Insert atleast three rows in each table.
 6. Read all the contents of each table.
 7. Update the name of the employee with empid 101 as 'Anjali'
 8. Delete the record of dependent of the employee whose empid is 102.
 9. Delete a specific Dependent named 'Kiran' of Employee 101.
-

II. Write and execute CQL queries to retrieve specific data from Cassandra tables

1. Display dependentname and age of all dependents in the company.
 2. Display the names of all dependents of employee with ID 101.
 3. List the Ids of employees who have dependents with age greater than 6.
 4. Display the `dependent` table. But, limit the number of results returned to one record.
 5. Display the details of all the dependents of an employee whose empid is 101 in the ascending order of dependentname.
 6. Insert a new column 'gender' in dependent table. Update the values in each record also.
 7. Drop the column 'gender' from dependent table.
 8. Find Number of Dependents for an Employee whose id is 101
 9. Display id and name of employees who were born after 1st January 1995.
 10. Find the maximum age of the dependent of employee whose id is 101
-

ANSWERS

1. create keyspace company with replication={ 'class': 'SimpleStrategy', 'replication_factor': 1 };
2. describe keyspaces;
3. use company;
4. a. CREATE TABLE department (did INT PRIMARY KEY, dname TEXT, fund DECIMAL);
b. CREATE TABLE employee (eid INT PRIMARY KEY, ename TEXT, dob DATE, deptid INT);
c. CREATE TABLE dependent (empid INT, dependentname TEXT, age INT, PRIMARY KEY (empid, dependentname));
5. INSERT INTO department (did, dname, fund) VALUES (1, 'HR', 500000);
INSERT INTO department (did, dname, fund) VALUES (2, 'IT', 1200000);
INSERT INTO department (did, dname, fund) VALUES (3, 'Finance', 800000);

INSERT INTO employee (eid, ename, dob, deptid) VALUES (101, 'Asha', '1995-04-12', 1);
INSERT INTO employee (eid, ename, dob, deptid) VALUES (102, 'Rahul', '1993-08-20', 2);
INSERT INTO employee (eid, ename, dob, deptid) VALUES (103, 'Meena', '1990-01-05', 3);

```
INSERT INTO dependent (empid, dependentname, age) VALUES (101, 'Anu', 5);
INSERT INTO dependent (empid, dependentname, age) VALUES (101, 'Kiran', 8);
INSERT INTO dependent (empid, dependentname, age) VALUES (102, 'Lakshmi', 30);
```

```
6. select *from department;
   select *from employee;
   select *from dependent;
```

```
7.UPDATE employee SET ename = 'Anjali' WHERE eid = 101; //should use primary key in where clause
```

```
8. DELETE FROM dependent WHERE empid = 102;
```

```
9. DELETE FROM dependent WHERE empid = 101 AND dependentname = 'Kiran';
```

```
1. select dependentname,age from dependent;
```

```
2. select dependentname from dependent where empid=101;
```

```
3. select empid from dependent where age>6 allow filtering; //where clause accepts only '=' case without using 'allow filtering'.
```

```
4. select * from department limit 1; //returns the first row
```

```
5. select * from dependent where empid=101 order by dependentname;
```

```
6.alter table dependent add gender text;
   update dependent set gender='female' where empid=102 and dependentname='Lakshmi'; //need to give both empid (PartitionKey)and dependentname(clusteringkey) to update a row.
```

```
7. alter table dependent drop gender;
   select * from dependent;
```

```
8. SELECT COUNT(*) FROM dependent WHERE empid = 101;
```

```
9. SELECT eid, ename FROM employee WHERE dob > '1995-01-01' ALLOW FILTERING;
```

```
10. select max(age) from dependent where empid=101;
```